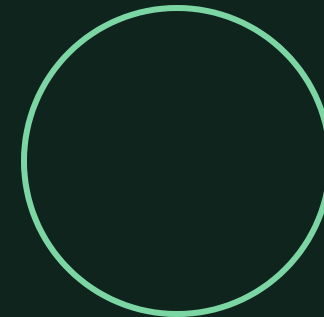


Trustworthy AI Software Modernization



Turning risky rewrites into measured migration workflows

Demo: Python to Rust with PyO3

Vision: language-agnostic modernization

Proof: tests + benchmarks

AI should help valuable software survive technological change.

The Problem



Important software often survives longer than the technology stack it was built on.

- Rewrites are expensive, slow, and risky
- Behavior is hidden in edge cases, not documented
- Small organizations often lack the time or specialists to modernize systems
- AI can write code, but unvalidated AI rewrites are hard to trust

The modernization gap: AI can accelerate migration, but only transparent, tested, and measurable workflows can make it safe and accessible for everyone.

Why Modernize Code?

Migration is not about chasing a newer language. It is about making useful software cheaper, safer, and easier to keep alive.

Price

Moving from expensive proprietary ecosystems, such as MATLAB, to open ecosystems like Python can lower licensing barriers for students, researchers, nonprofits, and small companies.

Sustainability

Moving performance-critical Python code to Rust can reduce CPU time, which can lower energy use and cloud cost for repeated workloads.

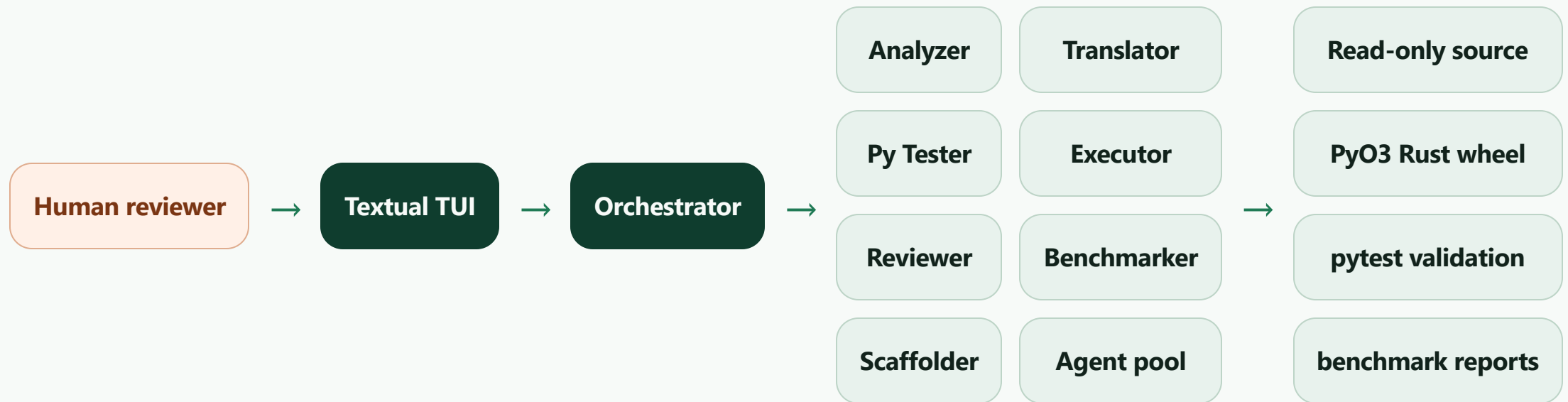
Longevity

Updating older code to newer languages, runtimes, and package formats extends

Security

Modernizing can remove risky dependencies and use safer defaults, for

System Architecture



Why this matters: agents work through constrained tools, scoped outputs, quality gates, and human checkpoints.

Migration Workflow

1. CAPTURE BEHAVIOR

Analyze the Python project and generate pytest that documents the current behavior.

2. HUMAN REVIEW

Reviewer summarizes coverage, risk, and suggested focus before approval.

3. SCAFFOLD RUST

Create a PyO3/maturin project that preserves the Python import surface.

4. TRANSLATE BEHAVIOR

Implement Rust code behind the same Python-facing API.

5. VALIDATE WHEEL

Build with maturin, install the wheel, and run the same pytest suite.

6. MEASURE IMPACT

Discover benchmark cases, compare isolated Python vs Rust wheels, and produce CSV, text, metadata, and graph artifacts.

Why Python to Rust?



Python is accessible and widely used. Rust is fast, memory-safe, and efficient.

That makes Python-to-Rust a strong first case study:

- Keep the Python API that users already know
- Move performance-critical internals to Rust
- Build a distributable wheel with PyO3 and maturin
- Validate that behavior still matches the original package
- Measure whether the migration actually improves performance

The point is not only Rust. The point is trustworthy modernization.

Our Approach

We built an agentic migration pipeline that treats translation as an engineering process, not a one-shot prompt.

Preserve behavior first

Generate pytest suites against the original Python project before changing the implementation.

Reimplement with guardrails

Translate into a Rust PyO3 extension while preserving the Python-facing API.

Validate with real tools

Run linting, Rust quality gates, maturin builds, installed-wheel pytest, and fix loops.

Measure the result

Benchmark isolated Python and Rust wheel installs and write performance reports to measurement artifacts.

Safety rule: the original source project is read-only. Generated tests, Rust code, and

Why Judges Should Care

Most AI coding demos show generation. This project shows preservation.

Trust

Behavior is encoded as tests before implementation is replaced.

Transparency

Human reviewers see plans, generated artifacts, risks, and summaries.

Accountability

Migration success is checked by compilers, test runners, and benchmark reports.

Access

Small teams get a guided modernization workflow normally reserved for large engineering organizations.

Demo Story

In the demo, we can show:

1. Start from an existing Python project
2. Generate a migration plan and behavior-preserving pytest suite
3. Review the plan and tests before translating implementation
4. Scaffold and implement a Rust PyO3 package
5. Build and install the Rust wheel with maturin
6. Run the same pytest suite against the migrated wheel
7. Produce Python vs Rust benchmark measurements from isolated wheel installs

Narrative: the user stays in control while agents do the repetitive migration, validation, and repair work.

Benchmarking Proof

The latest workflow does not just claim the Rust migration is faster. It measures it.

Auto-discovery

Benchmark cases can come from custom suites, known generators, API signatures, or pytest call patterns.

LLM fallback

If automatic discovery is not enough, the Benchmarking can inspect the project and write a benchmark suite.

Fair comparison

Python source and Rust PyO3 wheels are installed into isolated targets before timing.

Real outputs

Reports include raw runs, summaries, metadata, and graphs for latency, variance, resources, and speedup.

Beyond Python to Rust

Python to Rust is the first proof point, not the limit.

What stays the same

Analyze, test, review,
translate, validate, repair,
measure.

What changes

Language prompts,
project scaffold, package
tools, test runner, and
benchmark adapter.

What this enables

Language-agnostic
migration profiles for
different modernization
goals.

Examples:

- JavaScript to TypeScript for safer maintainability
- Python to Go for deployment simplicity
- Java to Kotlin for JVM modernization
- R or MATLAB to Python for scientific software longevity

Current State

Implemented in the repo today:

Workflow

Six-step pipeline: tests, review, PyO3 translation, review, wheel validation, benchmarking.

Agents

Specialized roles for analysis, pytest generation, review, scaffolding, translation, execution, and measurement.

Quality gates

flake8, mypy, baseline pytest, cargo fmt, cargo clippy, maturin build, installed-wheel pytest.

Measurement

Benchmarks compare isolated Python and Rust wheel installs across input tiers and write reports to measurements.

Next step: formalize language profiles so source language, target language, packaging, validation, and measurement become configurable.

Impact



Remember why modernization matters:

Price

Open and efficient ecosystems reduce licensing, infrastructure, and operational costs.

Sustainability

Faster code can use less CPU, lowering energy use for repeated workloads.

Longevity

Updating old code keeps important systems usable for future teams.

Access

Smaller organizations can modernize without needing a large rewrite budget.

Goal: make trustworthy modernization available to the teams that need it most.

Ask / Next Steps

We are looking for feedback on:

- Which language pairs would create the most social impact?
- What evidence would make users trust an AI-assisted migration?
- Which benchmark metrics matter most: latency, cost, energy, memory, or package size?
- How should human review be designed for non-expert maintainers?

Hackathon goal: prove trustworthy Python-to-Rust modernization, then generalize the pattern into migration profiles for many language pairs.

Closing



AI should help software survive technological change.

Not just generate new code, but preserve behavior, reduce migration risk, and extend the life of important systems.

Agentic Code Migration for Good

Test-driven. Human-reviewed. Benchmarked. Built for trustworthy modernization.